



TypeScript para Angular

Tipado, componentes, servicios, formularios y consumo de APIs

TYPESCRIPT

`const total: number`

JavaScript con tipos y mejores herramientas

ANGULAR

`@Component({...})`

Aplicaciones web basadas en componentes

INTEGRACIÓN

`Model → Service → UI`

Código más claro, seguro y mantenible

TIPOS SEGUROS

COMPONENTES

APLICACIÓN REAL

ESTUDIANTE

Juan Andrés Suárez Gaviria

BOOTCAMP

Desarrollo web fullstack

FECHA

22 de septiembre de 2026

Investigación completa | Fundamentos aplicados al desarrollo con Angular

¿Qué es TypeScript?

DEFINICIÓN

TypeScript es un lenguaje construido sobre JavaScript que agrega sintaxis para definir tipos. Su compilador revisa el código y lo transforma en JavaScript, que es el lenguaje que finalmente ejecuta el navegador.

¿Por qué Angular utiliza TypeScript?

ERRORES TEMPRANOS

Detecta incompatibilidades antes de ejecutar.

AUTOCOMPLETADO

El editor conoce propiedades y métodos.

MANTENIMIENTO

Los contratos hacen el código más entendible.

Del archivo .ts al navegador

1 Código TypeScript

Archivos .ts

2 Comprobación

Tipos y plantillas

3 Compilación

Angular CLI

4 JavaScript

Código optimizado

5 Navegador

Aplicación ejecutada

JavaScript y TypeScript

Aspecto	JavaScript	TypeScript
Tipos	Dinámicos durante la ejecución	Comprobados antes de ejecutar
Errores	Muchos aparecen al usar la aplicación	Varios se detectan al compilar
Editor	Ayuda basada en inferencias	Autocompletado y refactorización precisos
Ejecución	El navegador lo ejecuta directamente	Primero se transforma a JavaScript

TypeScript no reemplaza a JavaScript: lo amplía durante el desarrollo y después produce JavaScript compatible con la aplicación.

Tipos básicos e inferencia

El tipo describe qué valores puede guardar una variable y qué operaciones son válidas sobre ella.

Tipos más utilizados

Tipo	Ejemplo	Uso
string	let nombre: string = 'Ana'	Texto
number	let precio: number = 45000	Enteros y decimales
boolean	let activo: boolean = true	Estados verdadero o falso
Array<T>	let ids: number[] = [1, 2]	Colección de valores del mismo tipo
object	{ nombre: 'Ana', edad: 25 }	Datos agrupados por propiedades
null / undefined	let dato: string null	Ausencia controlada de un valor
void	function guardar(): void	Función que no devuelve un valor
unknown	let respuesta: unknown	Valor desconocido que debe comprobarse

Inferencia, uniones y tipos literales

INFERENCIA Y UNIONES

```
// TypeScript infiere string
let ciudad = "Medellín";

// Unión: admite dos posibilidades
let identificador: number | string;
identificador = 25;
identificador = "USR-25";

// Literal: limita los valores
type Estado = "activo" | "inactivo";
let estado: Estado = "activo";
```

ARREGLOS Y TUPLAS

```
const tecnologias: string[] = [
  "TypeScript",
  "Angular"
];

// Tupla: posición y tipo definidos
const coordenada: [number, number] = [6.24, -75.58];

// Arreglo de objetos tipado
const productos: Array<{ id: number; nombre: string }> = [
  { id: 1, nombre: "Teclado" }
];
```

Conviene evitar `any` porque desactiva la comprobación de tipos. Cuando el dato aún no se conoce, `unknown` obliga a validarlo antes de usarlo.

Interfaces y alias de tipos

CONTRATOS

Las interfaces y los alias de tipos describen la forma de los datos. En Angular son útiles para modelos, propiedades de componentes, formularios y respuestas recibidas desde una API.

Interface aplicada a un modelo

MODELS/PRODUCTO.MODEL.TS

```
export interface Producto {
  readonly id: number;
  nombre: string;
  precio: number;
  disponible: boolean;
  descripcion?: string;
  categoria: Categoria;
}

export type Categoria =
  | "tecnología"
  | "hogar"
  | "oficina";

const producto: Producto = {
  id: 1,
  nombre: "Teclado",
  precio: 180000,
  disponible: true,
  categoria: "tecnología"
};
```

Símbolo	Significado
readonly	No permite reasignar la propiedad
?	Propiedad opcional
	Unión de tipos
export	Permite importar el tipo

Interface o type

Opción	Se recomienda para	Ejemplo
interface	Objetos y contratos que pueden extenderse	interface Usuario { id: number }
type	Uniones, literales, tuplas y composiciones	type Rol = 'admin' 'cliente'

Importar y reutilizar

MÓDULOS DE TYPESCRIPT

```
import { Producto } from "../models/producto.model";

function calcularTotal(productos: Producto[]): number {
  return productos.reduce((total, item) => total + item.precio, 0);
}
```

Los tipos ayudan durante el desarrollo, pero no validan por sí solos los datos que llegan desde Internet. Las respuestas externas también deben comprobarse.

Funciones tipadas

DEFINICIÓN

TypeScript permite indicar el tipo de cada parámetro y del valor devuelto. Esto evita llamadas incorrectas y ayuda a comprender qué espera y qué produce una función.

Parámetros y valores de retorno

FUNCIONES

```
function aplicarDescuento(
  precio: number,
  porcentaje: number = 10
): number {
  return precio - precio * (porcentaje / 100);
}

function buscarProducto(id: number): Producto | undefined {
  return productos.find(producto => producto.id === id);
}

function mostrarMensaje(mensaje?: string): void {
  console.log(mensaje ?? "Sin mensaje");
}
```

Formas frecuentes

Forma	Sintaxis	Aplicación
Obligatorio	nombre: string	Siempre debe enviarse
Opcional	descripcion?: string	Puede omitirse
Predeterminado	cantidad: number = 1	Usa un valor si no se envía
Rest	...ids: number[]	Recibe varios argumentos
Callback	accion: (id: number) => void	Recibe otra función

Ejemplos dentro de Angular

EVENTOS Y ESTADO

```
seleccionar(producto: Producto): void {
  this.productoSeleccionado.set(producto);
}

eliminar(id: number): void {
  this.productos.update(lista =>
    lista.filter(item => item.id !== id)
  );
}
```

FUNCIÓN ASÍNCRONA

```
async function cargarConfiguracion(): Promise<Configuracion> {
  const respuesta = await fetch("/config.json");

  if (!respuesta.ok) {
    throw new Error("No fue posible cargar");
  }

  return respuesta.json() as Promise<Configuracion>;
}
```

Promise<T> representa un valor que estará disponible después. En Angular, HttpClient normalmente trabaja con Observable<T>.

Clases y programación orientada a objetos

Angular utiliza clases para componentes, servicios, directivas y otras piezas de la aplicación.

Clase con propiedades y métodos

PRODUCTO.TS

```
interface Identificable {
  id: number;
}

class Producto implements Identificable {
  constructor(
    public readonly id: number,
    public nombre: string,
    private costo: number,
    public precio: number
  ) {}

  obtenerGanancia(): number {
    return this.precio - this.costo;
  }
}

const teclado = new Producto(1, "Teclado", 120000, 180000);
console.log(teclado.obtenerGanancia());
```

Modificadores de acceso

Modificador	Acceso	Uso frecuente
public	Desde cualquier parte	Propiedades usadas por la plantilla
private	Solamente dentro de la clase	Detalles internos
protected	Clase y clases derivadas	Herencia
readonly	Lectura después de inicializar	Identificadores y dependencias

Ciclo de vida tipado en Angular

INICIO.COMPONENT.TS

```
import { Component, OnInit } from "@angular/core";

@Component({
  selector: "app-inicio",
  templateUrl: "./inicio.component.html"
})
export class InicioComponent implements OnInit {
  titulo: string = "Mi aplicación";

  ngOnInit(): void {
    console.log("Componente inicializado");
  }
}
```

Implementar OnInit es opcional, pero permite que TypeScript compruebe el nombre y la firma correcta de ngOnInit().

Genéricos, utilidades y comprobaciones

GENÉRICOS

Un genérico permite escribir estructuras reutilizables sin perder el tipo concreto. La letra T funciona como un espacio que se reemplaza al utilizar la estructura.

Respuesta genérica para una API

GENÉRICOS

```
export interface RespuestaApi<T> {
  datos: T;
  mensaje: string;
  correcto: boolean;
}

const respuesta: RespuestaApi<Producto[]> = {
  datos: [],
  mensaje: "Consulta correcta",
  correcto: true
};

function primero<T>(elementos: T[]): T | undefined {
  return elementos[0];
}
```

Tipos de utilidad

Utilidad	Resultado	Ejemplo
Partial<T>	Todas las propiedades son opcionales	Actualizar parcialmente
Pick<T, K>	Conserva solamente algunas propiedades	Resumen de producto
Omit<T, K>	Excluye propiedades elegidas	Crear sin id
Record<K, T>	Objeto con claves y valores definidos	Estados por identificador

Narrowing: comprobar antes de utilizar

TYPE GUARD

```
function mostrarId(id: number | string): string {
  if (typeof id === "number") {
    return id.toFixed(0);
  }

  return id.toUpperCase();
}
```

UTILIDADES

```
type ProductoNuevo = Omit<Producto, "id">;
type ProductoParcial = Partial<Producto>;

const cambio: ProductoParcial = {
  precio: 150000
};
```

Los genéricos son especialmente útiles con HttpClient, formularios y componentes reutilizables porque mantienen el tipo desde la fuente hasta la vista.

TypeScript dentro de un componente Angular

COMPONENTE

Un componente combina una clase TypeScript, una plantilla HTML y metadatos. La clase guarda el estado y los métodos; el decorador @Component explica a Angular cómo utilizarla.

Componente standalone y estado con signals

PRODUCTO-CARD.COMPONENT.TS

```
import { Component, computed, input, output, signal } from "@angular/core";
import { CurrencyPipe } from "@angular/common";
import { Producto } from "../models/producto.model";

@Component({
  selector: "app-producto-card",
  standalone: true,
  imports: [CurrencyPipe],
  templateUrl: "./producto-card.component.html"
})
export class ProductoCardComponent {
  producto = input.required<Producto>();
  eliminado = output<number>();
  cantidad = signal<number>(1);

  total = computed(() =>
    this.producto().precio * this.cantidad()
  );

  aumentar(): void {
    this.cantidad.update(valor => valor + 1);
  }
}
```

Qué aporta TypeScript

Elemento	Tipo declarado	Beneficio
producto	InputSignal<Producto>	La plantilla conoce sus propiedades
eliminado	OutputEmitterRef<number>	Solamente emite identificadores numéricos
cantidad	WritableSignal<number>	El estado conserva un tipo consistente
total	Signal<number>	El cálculo se actualiza automáticamente

PRODUCTO-CARD.COMPONENT.HTML

```
<article>
  <h2>{{ producto().nombre }}</h2>
  <p>{{ total() | currency:'COP' }}</p>
  <button (click)="aumentar()">Agregar unidad</button>
  <button (click)="eliminado.emit(producto().id)">Eliminar</button>
</article>
```

Servicios, inyección y HttpClient

Un servicio concentra lógica reutilizable. Angular lo entrega a los componentes mediante inyección de dependencias.

Servicio tipado para consumir una API

SERVICES/PRODUCTOS.SERVICE.TS

```
import { inject, Injectable } from "@angular/core";
import { HttpClient } from "@angular/common/http";
import { Observable } from "rxjs";
import { Producto } from "../models/producto.model";

@Injectable({ providedIn: "root" })
export class ProductosService {
  private readonly http = inject(HttpClient);
  private readonly url = "https://api.ejemplo.com/productos";

  listar(): Observable<Producto[]> {
    return this.http.get<Producto[]>(this.url);
  }

  crear(producto: Omit<Producto, "id">): Observable<Producto> {
    return this.http.post<Producto>(this.url, producto);
  }

  eliminar(id: number): Observable<void> {
    return this.http.delete<void>(this.url + "/" + id);
  }
}
```

Configurar e inyectar

APP.CONFIG.TS

```
import { ApplicationConfig } from "@angular/core";
import { provideHttpClient } from "@angular/common/http";

export const appConfig: ApplicationConfig = {
  providers: [provideHttpClient()]
};
```

COMPONENTE

```
export class ProductosComponent {
  private readonly servicio = inject(ProductosService);

  productos$ = this.servicio.listar();
}
```

Flujo de los datos

1 Componente

Solicita productos

2 Servicio

Ejecuta HttpClient

3 API

Devuelve JSON

4 Observable

Emite Producto[]

5 Plantilla

Muestra los datos

El genérico de HttpClient mejora el autocompletado, pero no verifica en tiempo de ejecución que el servidor realmente envíe esa estructura.

Formularios tipados y buenas prácticas

FORMULARIOS REACTIVOS

Los formularios reactivos de Angular permiten definir controles y validaciones desde TypeScript. Al estar tipados, el editor conoce la forma y los valores del formulario.

Formulario para crear un producto

PRODUCTO-FORM.COMPONENT.TS

```
import { Component, inject } from "@angular/core";
import { NonNullableFormBuilder, Validators } from "@angular/forms";

@Component({
  selector: "app-producto-form",
  templateUrl: "./producto-form.component.html"
})
export class ProductoFormComponent {
  private readonly fb = inject(NonNullableFormBuilder);

  formulario = this.fb.group({
    nombre: ["", [Validators.required]],
    precio: [0, [Validators.required, Validators.min(0)]],
    disponible: [true]
  });

  guardar(): void {
    if (this.formulario.invalid) return;
    const producto = this.formulario.getRawValue();
    console.log(producto);
  }
}
```

Práctica	Motivo
strict	Detecta más errores
Evitar any	Conserva seguridad
Modelos	Centralizan contratos
readonly	Evita reasignaciones
Tipos de retorno	Aclaran funciones
Archivos pequeños	Mejoran el orden

Organización de una aplicación

ESTRUCTURA

```
src/app/
|-- models/producto.model.ts
|-- services/productos.service.ts
|-- components/
|   |-- producto-lista/
|   |-- producto-card/
|   '-- producto-form/
|-- app.component.ts
'-- app.config.ts
```

1 Model

Define
Producto

2 Service

Consulta la API

3 Component

Maneja el
estado

4 Template

Presenta la
interfaz

Separar responsabilidades permite que cada archivo sea fácil de entender, probar y modificar.

Comandos esenciales

TERMINAL

```
npm install -g @angular/cli
ng new tienda-angular
cd tienda-angular
ng serve
ng generate component components/producto-lista
ng generate service services/productos
```

La versión de TypeScript debe ser compatible con la versión de Angular del proyecto. Lo más seguro es utilizar la configuración creada por Angular CLI.